

Taller Práctico 4: Red Neuronal en una GPU

Introducción

Este taller implementa una red neuronal artificial para el reconocimiento de dígitos escritos a mano, utilizando el conjunto de datos MNIST. El objetivo principal es demostrar cómo las operaciones matemáticas de una red neuronal pueden ejecutarse directamente en una GPU mediante kernels CUDA, aprovechando el paralelismo masivo que ofrece este tipo de hardware. Para ello se combinan dos herramientas: Keras para el entrenamiento del modelo, y Numba CUDA para la implementación manual del proceso de inferencia en la GPU.

Desarrollo

Celda 1: Verificación de la GPU

El primer paso consiste en confirmar que el entorno de ejecución cuenta con una GPU disponible. Se consulta la herramienta `nvidia-smi`, que reporta información del hardware gráfico instalado. Esto es indispensable antes de cualquier operación CUDA, ya que sin GPU el resto del notebook no puede ejecutarse correctamente. En este caso se confirmó la presencia de una GPU Tesla T4.

Celda 2: Verificación de Numba CUDA

Se importa la librería Numba y se verifica que su módulo CUDA puede comunicarse con la GPU detectada. Numba es el puente entre Python y CUDA: permite escribir kernels en Python con el decorador `@cuda.jit`, los cuales son compilados y ejecutados directamente en la GPU. Se confirma que CUDA está disponible y se imprime el nombre del dispositivo.

Celda 3: Carga de datos y entrenamiento

Se carga el conjunto de datos MNIST, que contiene 70.000 imágenes en escala de grises de dígitos del 0 al 9, cada una de 28x28 píxeles. Los valores de píxel se normalizan al rango $[0, 1]$

dividiéndolos entre 255.

Se define una red neuronal secuencial con tres capas: una capa de aplanamiento que convierte cada imagen de 28x28 en un vector de 784 valores, una capa densa de 128 neuronas con función de activación ReLU, y una capa de salida de 10 neuronas con función de activación Softmax, correspondiente a los 10 posibles dígitos.

El modelo se entrena durante 5 épocas con el optimizador Adam, alcanzando una precisión del 97.42 % sobre el conjunto de prueba.

Celda 4: Extracción de pesos

Una vez entrenado el modelo, se extraen los pesos y sesgos de cada capa densa. Estos parámetros representan el conocimiento aprendido por la red durante el entrenamiento. Se obtienen cuatro matrices: W_1 de forma (784, 128), b_1 de forma (128,), W_2 de forma (128, 10) y b_2 de forma (10,). Estos valores se utilizarán en las celdas siguientes para reimplementar manualmente la inferencia en GPU.

Celda 5: Definición de kernels CUDA

Se definen tres kernels CUDA usando el decorador `@cuda.jit` de Numba, siguiendo la misma metodología del taller anterior:

- **matmul_bias_kernel:** realiza la multiplicación entre el vector de entrada y la matriz de pesos, sumando el sesgo. Cada hilo de la GPU calcula un elemento del vector de salida de forma paralela.
- **relu_kernel:** aplica la función de activación ReLU a cada elemento del vector, reemplazando los valores negativos por cero. Cada hilo procesa un elemento independiente.
- **softmax_exp_kernel:** calcula la exponencial de cada elemento del vector de salida, paso previo a la normalización de Softmax.

Celda 6: Forward pass en GPU

Se implementa la función de inferencia completa utilizando los kernels definidos anteriormente. Dado una imagen de entrada, el proceso es el siguiente: primero se aplanan en un vector de 784 valores; luego pasa por la primera capa densa ejecutando `matmul_bias_kernel` seguido de `relu_kernel`; después pasa por la segunda capa densa con otro `matmul_bias_kernel`; finalmente se aplica Softmax en CPU sobre los 10 valores de salida, ya que operar 10 elementos no justifica el overhead de un kernel adicional. El resultado es un vector de probabilidades donde cada posición corresponde a un dígito.

Celda 7: Prueba con imágenes de MNIST

Se seleccionan cinco imágenes aleatorias del conjunto de prueba y se pasan por el forward pass implementado en GPU. Para cada imagen se muestra el dígito real, la predicción de la red y el nivel de confianza. Las cinco predicciones fueron correctas, con niveles de confianza superiores al 99 %, lo que valida que la implementación manual en CUDA produce resultados equivalentes al modelo original de Keras.

Celda 8: Prueba con imagen guardada como archivo

Para simular un caso de uso real, se toma una imagen del conjunto de prueba, se guarda en disco como archivo PNG y se vuelve a cargar como si fuera una imagen externa. Se convierte a escala de grises, se redimensiona a 28x28 píxeles y se normaliza. La red predijo correctamente el dígito 4 con una confianza del 99.85 %, demostrando que el flujo funciona con imágenes provenientes de fuentes externas.

Celda 9: Prueba con imagen propia

Se habilita la carga de una imagen creada manualmente por el usuario en un programa de edición. La imagen debe tener fondo negro y el dígito dibujado en blanco, siguiendo el mismo formato que las imágenes de MNIST. La imagen se convierte a escala de grises, se redimensiona a 28x28 y se normaliza antes de pasar por el forward pass. La red reconoció correctamente el dígito dibujado, confirmando que el modelo generaliza más allá del conjunto de datos original.

Conclusiones

Este taller demostró que es posible descomponer una red neuronal entrenada en alto nivel y reimplementar su proceso de inferencia directamente en la GPU mediante kernels CUDA. Cada operación matemática de la red, como la multiplicación matricial y la activación ReLU, se ejecutó en paralelo aprovechando los miles de núcleos de la GPU Tesla T4. Los resultados obtenidos fueron equivalentes a los del modelo original, con precisiones superiores al 97 % y tiempos de respuesta inmediatos. Este enfoque permite entender en profundidad qué ocurre dentro de una red neuronal a nivel computacional, más allá de las abstracciones que ofrecen los frameworks de alto nivel.